

# Pandas DataFrame Lecture Notes

Session 17 - Data Science Mentorship Program (DSMP) 2022-23

**Speaker:** Nitish Singh (CampusX)

## 1. Introduction to Pandas DataFrame

A **DataFrame** is a 2-dimensional labeled data structure with columns of potentially different types. You can think of it like a spreadsheet or SQL table.

### Key Concepts:

- **Structure:** It consists of rows and columns.
- **Relationship with Series:**
  - A single column in a DataFrame is a **Series**.
  - A single row in a DataFrame is also treated as a **Series**.
  - A DataFrame is essentially a collection of Series objects sharing the same index.

## 2. Creating DataFrames

### A. From 2D Lists

To create a DataFrame from a list of lists, you provide the data and optionally specify column names.

```
import pandas as pd
```

```
# Data representing [IQ, Marks, Package]
```

```
student_data = [  
    [100, 80, 10],  
    [90, 70, 7],  
    [120, 100, 14],  
    [80, 50, 2]  
]
```

```
df = pd.DataFrame(student_data, columns=['IQ', 'Marks', 'Package'])  
print(df)
```

### B. From Dictionaries

Each key in the dictionary becomes a column name, and the value (a list) becomes the column data.

```
student_dict = {  
    'IQ': [100, 90, 120, 80],  
    'Marks': [80, 70, 100, 50],  
    'Package': [10, 7, 14, 2]  
}  
  
df = pd.DataFrame(student_dict)
```

## C. From CSV Files

Commonly used for real-world datasets.

```
movies = pd.read_csv('bollywood_movies.csv')  
ipl = pd.read_csv('ipl-matches.csv')
```

# 3. Important Attributes & Methods

## Attributes

- `df.shape`: Returns a tuple representing the dimensionality (rows, columns).
- `df.dtypes`: Returns the data type of each column.
- `df.index`: Returns the index (row labels) of the DataFrame.
- `df.columns`: Returns the column labels.
- `df.values`: Returns a NumPy representation of the DataFrame (2D array).

## Exploration Methods

- `df.head(n)`: Shows the first `n` rows.
- `df.tail(n)`: Shows the last `n` rows.
- `df.sample(n)`: Shows `n` random rows (useful for checking data bias).
- `df.info()`: Provides a high-level summary (non-null counts, memory usage, dtypes).
- `df.describe()`: Provides statistical summary (mean, std, min, max, quartiles) for numerical columns.

## Data Integrity

- `df.isnull().sum()`: Shows the count of missing values in each column.
- `df.duplicated().sum()`: Shows the number of duplicate rows.

# 4. Mathematical Functions & Axis

Pandas allows performing math across the DataFrame. The axis parameter is crucial:

- **axis=0 (Default)**: Column-wise operation.

- **axis=1:** Row-wise operation.

```
# Column-wise sum  
df.sum()
```

```
# Row-wise sum  
df.sum(axis=1)
```

```
# Other functions: mean(), median(), std(), var(), min(), max()
```

## 5. Selecting Data from DataFrames

### Selecting Columns

- **Single Column:** `df['column_name']` (Returns a Series).
- **Multiple Columns (Fancy Indexing):** `df[['col1', 'col2']]` (Returns a DataFrame).

### Selecting Rows

- **iloc (Index Location):** Uses integer positions.
  - `df.iloc[0]`: First row.
  - `df.iloc[0:5]`: First 5 rows.
  - `df.iloc[[0, 5, 10]]`: Specific rows.
- **loc (Label Location):** Uses index labels.
  - `df.loc['label']`: Row with specific label.

### Selecting Both

- `df.iloc[rows_index, columns_index]`
- `df.loc[rows_labels, columns_labels]`

## 6. Filtering (Boolean Indexing)

This is the most powerful way to extract specific subsets of data.

### Examples:

#### 1. Find all IPL final winners:

```
ipl[ipl['match_number'] == 'Final'][['season', 'winning_team']]
```

#### 2. Find matches with Super Overs:

```
ipl[ipl['super_over'] == 'Y'].shape[0]
```

### 3. Multiple Conditions (CSK wins in Kolkata):

# Use & for AND, | for OR

```
ipl[(ipl['city'] == 'Kolkata') & (ipl['winning_team'] == 'Chennai Super Kings')]
```

### 4. Complex String Filtering (Action movies with rating > 7.5):

# Method using .str.contains()

```
action_movies = movies[(movies['genres'].str.contains('Action')) & (movies['imdb_rating'] > 7.5)]
```

## 7. Data Manipulation

### Renaming Columns

```
df.rename(columns={'Marks': 'Percent', 'Package': 'LPA'}, inplace=True)
```

### Adding New Columns

- **Constant value:** `df['Country'] = 'India'`

- **Derived value:** `python`

**Extracting the first actor as Lead Actor**

**`df['Lead Actor'] =`**

**`df['Actors'].str.split('|').str.get(0)`**

### Memory Optimization (astype)

Changing data types can significantly reduce RAM usage.

- `df['col'].astype('int32')`
- `df['col'].astype('category')`: Highly effective for columns with repetitive text values (e.g., Team names, City).

# Example of optimization

```
ipl['season'] = ipl['season'].astype('category')
```

```
ipl['team1'] = ipl['team1'].astype('category')
```

## 8. Summary of Functions

| Function                          | Purpose                         |
|-----------------------------------|---------------------------------|
| <code>pd.read_csv()</code>        | Load data from CSV              |
| <code>df.info()</code>            | Memory and null value summary   |
| <code>df.describe()</code>        | Statistical summary             |
| <code>df.iloc</code>              | Integer-based slicing           |
| <code>df.loc</code>               | Label-based slicing             |
| <code>df.astype()</code>          | Change data types               |
| <code>df.drop_duplicates()</code> | Remove duplicate rows           |
| <code>df.dropna()</code>          | Remove rows with missing values |

**Note:** Always use `inplace=True` or reassign the variable (`df = df.method()`) if you want to save the changes to the DataFrame.